

CLAUDE_CODE_BUILD_INSTRUCTIONS.md

Claude Code Build Instructions For GSpaceAi

You are Claude Code.

Build a brand-new production-ready MVP web app for GSpaceAi from scratch.

Do not reference or reuse any previous version of the app.

Do not assume any existing codebase.

Start clean.

The app must implement the product described in `GSPACEAI_PRODUCT_SPEC.md`.

Before writing code, read the full product specification and create an implementation plan.

Primary Build Objective

Build a real working MVP web app that allows a customer to:

1. Visit the app.
2. Enter their name.
3. Enter business details.
4. Complete a conversational software consolidation audit.
5. Receive a free Platform Consolidation Snapshot PDF.
6. Pay \$29 externally through Stripe.
7. Return and verify payment.
8. Receive the Recommendations Report PDF.
9. Pay \$79 externally through Stripe.
10. Return and verify payment.
11. Receive the Implementation Guide + SOP Book PDF.
12. Pay \$159 externally through Stripe.
13. Return and verify payment.
14. Receive thank-you message and next steps.

This is not a demo-only mockup.

It should be functional, modular, and ready for real customer testing.

Recommended Tech Stack

Use a clean modern stack suitable for a fast MVP.

Preferred stack:

- Next.js
- React
- TypeScript
- Tailwind CSS
- API routes/server actions
- Gemini API for AI reasoning/generation
- Server-side PDF generation
- Config-driven product/payment ladder
- Google Apps Script endpoint or verification adapter for payment verification
- Local/session persistence for MVP, with an easy path to database persistence later

If another stack is already selected by the environment, explain why and adapt.

Do not overbuild.

Prioritize:

- clear state management
- reliable payment gating
- reliable report generation
- clean UI
- modular files
- easy future expansion

Architecture Requirement

The app must separate:

1. UI components
2. workflow state management
3. product/payment configuration
4. AI audit logic
5. report content generation
6. PDF generation
7. payment verification
8. report gating rules
9. environment variables
10. product specification docs

Do not mix all logic inside one component.

Required Project Files

Create a structure similar to this:

```
/
  CLAUDE.md
  .env.example
  README.md

  /docs
    GSPACEAI_PRODUCT_SPEC.md
    CLAUDE_CODE_BUILD_INSTRUCTIONS.md
    REPORT_TEMPLATES.md
    PAYMENT_FLOW.md
    GOOGLE_WORKSPACE_CAPABILITY_LIBRARY.md
    COMMON_PLATFORM_REPLACEMENTS.md
    REPORT_STYLE_NOTES.md

  /src
    /app
      page.tsx
      layout.tsx
    /api
      /chat
        route.ts
      /verify-payment
        route.ts
      /generate-report
        route.ts

  /components
    ChatInterface.tsx
    MessageBubble.tsx
    OfferCard.tsx
    PaymentVerificationModal.tsx
    ReportViewer.tsx
    ProgressIndicator.tsx
    LoadingState.tsx
    ErrorMessage.tsx

  /lib
    aiAuditLogic.ts
    workflowState.ts
    productConfig.ts
    paymentVerification.ts
```

```
reportGeneration.ts
pdfGeneration.ts
scoring.ts
savingsEstimator.ts
types.ts
constants.ts

/styles
globals.css
```

Adjust the file structure as needed for the framework, but preserve the separation of concerns.

CLAUDE.md Requirement

Create a `CLAUDE.md` file in the project root.

It must explain:

- what GSpaceAi is
- product positioning
- core workflow
- payment gating rules
- report generation rules
- coding standards
- state management rules
- testing commands
- environment variables
- important safeguards

This file should help future Claude Code sessions understand the project.

Environment Variables

Create `.env.example` with:

```
GEMINI_API_KEY=PASTE_GEMINI_API_KEY_HERE

STRIPE_RECOMMENDATIONS_LINK=PASTE_29_PAYMENT_LINK_HERE
STRIPE_IMPLEMENTATION_LINK=PASTE_79_PAYMENT_LINK_HERE
STRIPE_DONE_WITH_YOU_LINK=PASTE_159_PAYMENT_LINK_HERE

PAYMENT_VERIFICATION_ENDPOINT=PASTE_GOOGLE_APPS_SCRIPT_WEB_APP_URL_HERE
```

```
NEXT_PUBLIC_APP_NAME=GSpaceAi
NEXT_PUBLIC_COMPANY_NAME=GSpace Solutions
NEXT_PUBLIC_SUPPORT_EMAIL=support@gspacesolutions.org
```

Do not hardcode secrets.

Do not commit real API keys.

Use placeholders where needed.

Product Configuration

Create a config-driven product ladder.

Suggested product config:

```
export const PRODUCTS = {
  recommendations_report: {
    key: "recommendations_report",
    name: "Recommendations Report",
    price: 29,
    paymentLinkEnvKey: "STRIPE_RECOMMENDATIONS_LINK",
    expectedPaymentTrackerValue: "recommendations_report",
    unlockedDeliverable: "recommendations_report"
  },
  implementation_guide_sop: {
    key: "implementation_guide_sop",
    name: "Implementation Guide + SOP Book",
    price: 79,
    paymentLinkEnvKey: "STRIPE_IMPLEMENTATION_LINK",
    expectedPaymentTrackerValue: "implementation_guide_sop",
    unlockedDeliverable: "implementation_guide_sop"
  },
  done_with_you: {
    key: "done_with_you",
    name: "Done-With-You Implementation",
    price: 159,
    paymentLinkEnvKey: "STRIPE_DONE_WITH_YOU_LINK",
    expectedPaymentTrackerValue: "done_with_you",
    unlockedDeliverable: "done_with_you_confirmation"
  }
};
```

All payment links must be configurable.

The app should not require code changes to update Stripe payment links.

Workflow State Machine

Implement explicit state management.

Do not rely on vague chat text to infer workflow state.

Required stages:

```
type WorkflowStage =  
  | "intro"  
  | "collect_name"  
  | "collect_business_basics"  
  | "audit_in_progress"  
  | "free_report_generating"  
  | "free_report_ready"  
  | "recommendations_payment_pending"  
  | "recommendations_email_requested"  
  | "recommendations_verifying"  
  | "recommendations_verified"  
  | "recommendations_report_generating"  
  | "recommendations_report_ready"  
  | "implementation_payment_pending"  
  | "implementation_email_requested"  
  | "implementation_verifying"  
  | "implementation_verified"  
  | "implementation_report_generating"  
  | "implementation_report_ready"  
  | "done_with_you_payment_pending"  
  | "done_with_you_email_requested"  
  | "done_with_you_verifying"  
  | "done_with_you_verified"  
  | "complete";
```

State transitions must be explicit and guarded.

Critical Payment Logic

Payment verification must NEVER advance the customer to the next tier unless the purchased deliverable has been generated, saved, and displayed.

Correct flow:

Payment verified

- generate purchased deliverable
- save purchased deliverable
- display purchased deliverable
- only then show next offer

Incorrect flow:

Payment verified

- skip deliverable generation
- show next offer

This is the most important workflow safeguard.

Do not allow this bug.

Payment Verification Requirements

Build a payment verification adapter.

The first implementation should support verifying against a Google Apps Script web app endpoint that reads a Google Sheet payment tracker.

Design it so the verification method can later be replaced with Stripe webhook/database verification.

Verification request should include:

```
{
  "email": "customer@example.com",
  "productKey": "recommendations_report"
}
```

Verification response should support:

```
{
  "verified": true,
  "email": "customer@example.com",
  "productKey": "recommendations_report",
  "amount": 29,
  "status": "paid",
  "timestamp": "2026-06-07T12:00:00Z"
}
```

If no verified payment is found, the app must not unlock anything.

If a payment exists for the email but not the requested product, the app must not unlock anything.

If verification endpoint is missing or fails, show a user-friendly error.

Payment Error Messages

If no verified payment is found:

“Payment could not be verified yet. Please confirm the email used at checkout and try again. If you just paid, wait a moment and click verify again.”

If the wrong product was found:

“We found a payment for this email, but not for this specific report. Please confirm you selected the correct offer and try again.”

If email is missing:

“Please enter the email address used at checkout so we can verify your payment.”

If the verification service fails:

“We could not complete payment verification right now. Please try again in a moment.”

Audit Data Model

Create TypeScript types for:

```
type UserProfile = {
  name: string;
  email?: string;
  businessName: string;
  businessType: string;
  businessSize?: string;
  currentGoogleWorkspaceUsage?: string;
};

type SoftwareInventoryItem = {
  name: string;
  category: string;
```



```

    estimatedMonthlyCost?: number;
    usedFor?: string;
    importance?: "Low" | "Medium" | "High" | "Critical";
    replacementPotential?: "Low" | "Medium" | "High";
    recommendedAction?: "Keep" | "Replace" | "Consolidate" | "Automate" |
    "Investigate";
    googleWorkspaceAlternative?: string;
    notes?: string;
};

type AuditState = {
  answers: Array<{
    question: string;
    answer: string;
    timestamp: string;
  }>;
  softwareInventory: SoftwareInventoryItem[];
  workflowInventory: any[];
  bottlenecks: string[];
  manualTasks: string[];
  googleWorkspaceOpportunities: string[];
  automationOpportunities: string[];
  consolidationOpportunities: string[];
  risks: string[];
  recommendations: string[];
  gspaceConsolidationScore: number;
  estimatedMonthlySoftwareSpend: number;
  estimatedReplaceableMonthlySpend: number;
  estimatedAnnualSavings: number;
};

```

Add stronger types as the implementation develops.

Audit Conversation Logic

The audit must be conversational.

It should ask one question at a time.

It should not feel like a long form.

Opening flow:

1. Brief intro.
2. Ask name.

3. Ask business name.
4. Ask business type/basic details.
5. Begin audit.

Very brief intro example:

“Hi, I’m GSpaceAi. I’ll help you find software you may be paying for twice and show where Google Workspace could potentially replace or simplify your current tools. First, what’s your name?”

The AI should then proceed naturally.

Required early discovery areas:

1. business type
2. services/products
3. current Google Workspace usage
4. current paid tools outside Google Workspace
5. estimated cost of each tool
6. lead/customer intake
7. scheduling
8. customer/project/job tracking
9. communication
10. document management
11. reporting
12. manual repetitive tasks
13. duplicate entry
14. tools they suspect are underused

The AI should ask follow-up questions when needed.

AI System Prompt Requirements

Create a dedicated AI system prompt file or function.

It must instruct Gemini to act as:

- a Google Workspace consolidation consultant
- a small business operations advisor
- a software stack analyst
- an automation opportunity detector

It should prioritize:

1. software overlap
2. subscription waste
3. Google Workspace underutilization

4. consolidation opportunities
5. automation opportunities
6. workflow inefficiencies

It must not:

- force every tool into Google Workspace
 - make guaranteed savings claims
 - provide fake exact pricing when unknown
 - overpromise implementation simplicity
 - give the full paid roadmap inside the free audit
-

GSpace Consolidation Score

Implement scoring logic in `scoring.ts`.

The score range is 0 to 100.

Factors:

- number of paid software tools
- estimated monthly software spend
- number of tools with high replacement potential
- Google Workspace underutilization
- number of manual tasks
- number of duplicate data entry points
- number of disconnected workflows
- number of reporting gaps

Suggested scoring model for MVP:

- software count contribution: up to 20 points
- estimated spend contribution: up to 20 points
- replacement potential contribution: up to 25 points
- Google Workspace underutilization contribution: up to 15 points
- manual work contribution: up to 10 points
- reporting/process fragmentation contribution: up to 10 points

Clamp score between 0 and 100.

Label score:

0–25: Minimal Opportunity
26–50: Moderate Opportunity
51–75: Strong Opportunity
76–100: High Opportunity

The score must be explainable.

Savings Estimator

Implement savings estimation in `savingsEstimator.ts`.

Calculate:

- estimated monthly software spend
- estimated replaceable monthly spend
- estimated annual savings
- number of potential replacement opportunities
- number of potential consolidation opportunities
- number of automation opportunities

Use user-provided costs when available.

If costs are unknown, either:

1. ask the user for estimated cost, or
2. use conservative placeholder estimates and label them clearly as estimates.

Do not guarantee savings.

Use language like:

- "Estimated potential savings"
 - "Potential replaceable spend"
 - "Based on the tools and usage described"
-

Report Generation

Create modular report generation functions.

Required reports:

1. `platform_consolidation_snapshot`
2. `recommendations_report`
3. `implementation_guide_sop`

Each report should have:

- structured content generation
- PDF rendering

- downloadable PDF URL or blob
- displayed summary in app
- saved/generated status in state

Do not generate all paid reports upfront.

Generate reports only when unlocked.

Report Naming

Use these exact report names:

Free:

Platform Consolidation Snapshot

\$29:

Recommendations Report

\$79:

Implementation Guide + SOP Book

Free Report Template

Platform Consolidation Snapshot sections:

1. Cover Page
2. Business Snapshot
3. Current Software Inventory
4. Estimated Monthly Software Spend
5. Google Workspace Usage Summary
6. GSpace Consolidation Score
7. Potential Consolidation Opportunities
8. Potential Automation Opportunities
9. Estimated Savings Range
10. High-Level Findings
11. Recommended Next Step: Recommendations Report

This report should provide real value.

It should not reveal detailed implementation instructions.

It should not include the full 30/60/90 roadmap.

Recommendations Report Template

Recommendations Report sections:

1. Cover Page
2. Executive Summary
3. Business Snapshot
4. Software Stack Review
5. GSpace Consolidation Score
6. Platform Replacement Analysis
7. Keep / Replace / Consolidate / Automate Table
8. Google Workspace Consolidation Opportunities
9. Workflow Bottlenecks
10. Manual Work Reduction Opportunities
11. Recommended Lightweight Systems
12. Quick Wins
13. Medium Complexity Projects
14. Advanced Opportunities
15. Estimated Monthly Savings
16. Estimated Annual Savings
17. Priority Matrix
18. Recommended 30/60/90 Day Roadmap
19. Recommended Next Step: Implementation Guide + SOP Book

Include useful tables.

Make recommendations specific to the customer.

Avoid generic filler.

Implementation Guide + SOP Book Template

Implementation Guide + SOP Book sections:

1. Cover Page
2. Implementation Overview
3. Recommended System Architecture
4. Required Google Workspace Components
5. Build Sequence
6. Step-by-Step Implementation Plan
7. Data Structure Recommendations
8. Automation Opportunities

9. Suggested Apps Script Functions
10. Gemini Usage Opportunities
11. Owner/Admin SOPs
12. Team/User SOPs
13. Maintenance SOPs
14. Testing Checklist
15. Launch Checklist
16. Recommended Next Step: Done-With-You Implementation

This report should be practical and implementation-oriented.

PDF Requirements

PDFs must be:

- branded as GSpaceAi / GSpace Solutions
- professional
- clean
- readable
- sectioned with headings
- table-friendly
- downloadable
- viewable from app
- generated server-side if possible

Avoid:

- walls of text
 - generic repetitive paragraphs
 - broken formatting
 - unstyled raw output
-

App UI Requirements

Build a simple, trustworthy interface.

Required components:

- landing/chat page
- chat/audit interface
- message bubbles
- progress indicator
- software inventory summary if useful
- report display area
- offer card

- payment link button
- verify payment button
- payment verification modal/form
- PDF download button
- loading states
- error states

The UI should feel:

- clean
- modern
- consultant-like
- easy to use
- not gimmicky

Do not overbuild dashboards for MVP.

Landing Page Requirements

The landing page should use the software consolidation positioning.

Main headline:

Stop Paying Twice For Software

Subheadline:

You already pay for Google Workspace. Discover which tools you're paying for that may already be replaceable using systems built inside Google Workspace.

CTA:

Start Free Assessment

Supporting bullets:

- Find overlapping software subscriptions
 - Estimate potential monthly savings
 - Discover Google Workspace replacement opportunities
 - Get your GSpace Consolidation Score
-

Offer Card Copy

\$29 Recommendations Report

Headline:

Unlock Your Personalized Replacement Roadmap

Copy:

Your free snapshot shows where software overlap may exist. The Recommendations Report gives you the full roadmap for which tools to keep, replace, consolidate, or automate using Google Workspace.

Button:

Get Recommendations Report - \$29

\$79 Implementation Guide + SOP Book

Headline:

Turn Your Roadmap Into A Build Plan

Copy:

Get the step-by-step implementation guide, recommended Google Workspace setup, automation ideas, and SOPs for putting your recommendations into action.

Button:

Get Implementation Guide + SOP Book - \$79

\$159 Done-With-You Implementation

Headline:

Want Help Implementing This?

Copy:

GSpace Solutions can help you review the roadmap and begin implementing the recommended systems with you.

Button:

Get Done-With-You Implementation - \$159

Persistence Requirements

For MVP, use the simplest reliable persistence available.

At minimum, do not lose the customer's stage/report immediately on basic UI actions.

Preferred options:

- localStorage/sessionStorage for MVP
- server/database persistence if simple and available

Make it easy to upgrade to database persistence later.

Do not let page refresh destroy paid unlock state if avoidable.

API Routes

Create API routes/server functions for:

`/api/chat`

Handles audit conversation and AI response.

`/api/generate-report`

Generates the requested report if allowed by state.

Must enforce report gating.

`/api/verify-payment`

Verifies customer payment for a specific product.

Must not unlock incorrect product.

Optional later route:

`/api/save-lead`

Captures user/audit/payment info for GSpace Solutions follow-up.

Report Gating Rules

Free report:

Can generate after audit has enough information.

Recommendations Report:

Can generate only after verified \$29 payment for `recommendations_report`.

Implementation Guide + SOP Book:

Can generate only after verified \$79 payment for `implementation_guide_sop`.

Done-With-You confirmation:

Can display only after verified \$159 payment for `done_with_you`.

No paid report should be generated before payment verification.

Done-With-You Completion Logic

After verified \$159 payment:

Do not generate another PDF.

Display:

“Thank you for purchasing Done-With-You Implementation. GSpace Solutions will review your audit and reports, then follow up with next steps.”

Include configurable support email.

Set workflow stage to `complete`.

Save customer information for follow-up if persistence exists.

Required Safeguards

1. Never unlock paid deliverables without verified payment.
 2. Never advance to next offer before generating and displaying the current purchased deliverable.
 3. Never show Implementation Guide before \$79 payment verification.
 4. Never show Done-With-You confirmation before \$159 payment verification.
 5. Save report generation output before displaying next offer.
 6. Keep payment verification product-specific.
 7. Keep report templates modular and editable.
 8. Keep AI instructions separate from app workflow logic.
 9. Make all payment links configurable.
 10. Make all report templates editable without rewriting app logic.
 11. Do not generate paid reports during free audit.
 12. Do not expose hidden paid roadmap inside the free report.
 13. Do not claim guaranteed savings.
 14. Do not force every software tool into Google Workspace.
-

Manual Testing Requirements

Create tests or a manual QA script covering:

1. New user starts assessment.
 2. User enters name.
 3. User enters business details.
 4. Audit proceeds one question at a time.
 5. Software inventory is captured.
 6. Google Workspace usage is captured.
 7. Free Platform Consolidation Snapshot generates.
 8. Recommendations Report offer appears.
 9. Failed \$29 verification does not unlock report.
 10. Successful \$29 verification generates Recommendations Report.
 11. Next offer appears only after Recommendations Report displays.
 12. Failed \$79 verification does not unlock Implementation Guide + SOP Book.
 13. Successful \$79 verification generates Implementation Guide + SOP Book.
 14. Next offer appears only after Implementation Guide + SOP Book displays.
 15. Failed \$159 verification does not show final thank-you.
 16. Successful \$159 verification shows final next steps.
 17. Wrong email fails verification.
 18. Correct email but wrong product fails verification.
 19. Duplicate verification does not duplicate reports.
 20. Refreshing page does not destroy workflow state if persistence exists.
 21. AI does not provide full roadmap in free report.
 22. AI does not guarantee exact savings.
-

Development Phases

Work in phases.

Phase 1: Planning

Read product spec.

Create architecture recommendation.

Before coding, explain:

- stack choice
- folder structure
- state management approach
- payment verification approach
- PDF generation approach
- AI orchestration approach
- persistence approach

Phase 2: Project Setup

Create base app.

Install needed dependencies.

Set up Tailwind or styling system.

Create environment file example.

Create docs folder.

Create CLAUDE.md.

Phase 3: Core Workflow

Build:

- workflow state machine
- user profile collection
- business basics collection
- audit chat interface
- local/session persistence

Phase 4: AI Audit Logic

Build:

- AI system prompt
- audit question flow
- answer capture
- software inventory extraction
- scoring inputs
- consolidation opportunity extraction

Phase 5: Scoring + Savings

Build:

- GSpace Consolidation Score
- savings estimator
- opportunity counters
- estimated software spend summary

Phase 6: Free Report

Build:

- Platform Consolidation Snapshot generation
- PDF rendering
- report viewer
- download button
- \$29 offer card

Phase 7: Payment Verification

Build:

- payment verification modal
- email capture
- verification API route
- Google Apps Script endpoint adapter
- failed verification handling
- product-specific validation

Phase 8: Paid Reports

Build:

- Recommendations Report generation after \$29 verification
- Implementation Guide + SOP Book generation after \$79 verification
- strict gating logic

- offer progression

Phase 9: Done-With-You Flow

Build:

- \$159 offer
- payment verification
- thank-you message
- next steps
- lead capture placeholder

Phase 10: QA + Bug Fixing

Run manual tests.

Fix state bugs.

Fix report gating bugs.

Fix PDF formatting issues.

Simplify where possible.

Output Expectations After Each Build Phase

After each phase, provide:

1. Files created or changed.
2. What was implemented.
3. How to test it.
4. Environment variables needed.
5. Known placeholders.
6. Remaining work.
7. Any risks or technical decisions.

Initial Instruction To Claude Code

Start by reading:

1. `GSPACEAI_PRODUCT_SPEC.md`
2. `CLAUDE_CODE_BUILD_INSTRUCTIONS.md`

Then do not write code immediately.

First provide:

1. Recommended architecture
2. Recommended tech stack
3. Folder structure
4. State management design
5. Payment verification design
6. PDF generation design
7. AI orchestration design
8. Persistence design
9. Build phases
10. Questions or blockers

After the architecture is approved, begin Phase 1 implementation.

Final Goal

The finished MVP should be a working customer-facing app for GSpaceAi.

It should help users discover software they may be paying for twice, estimate potential savings, generate a free Platform Consolidation Snapshot, and unlock paid PDF deliverables only after verified payment.

The app must be simple, reliable, and built for real-world testing.